

Operating systems

Virtual Memory

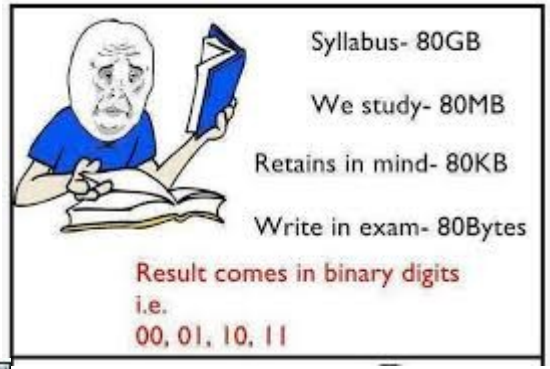
Content Lecture#12

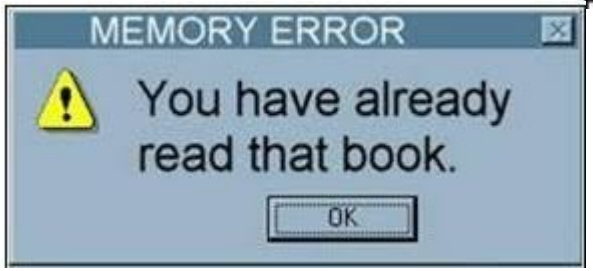
01	M
02	A
03	S
04	P
05	A

Lets take a Break

A programmer had a problem, so he used threds. Then he had two problems.









Virtual Memory

A

Address binding

is

Virtual Memory

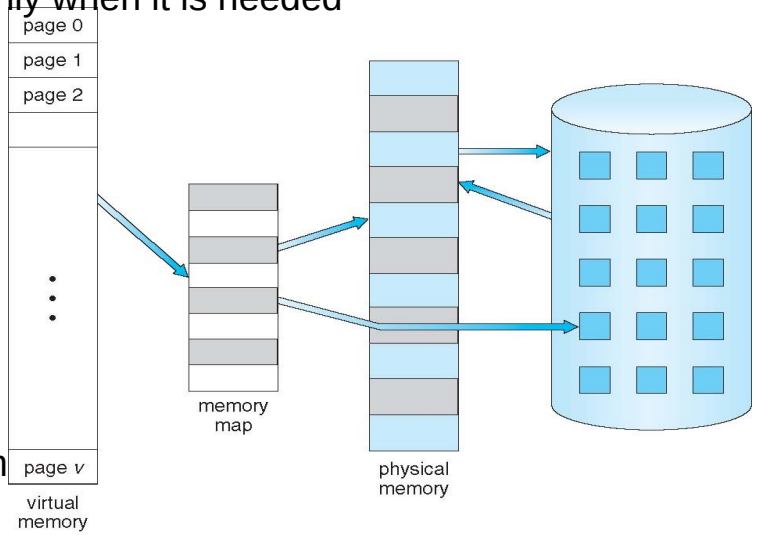
- Why we need virtual memory
  - The program size may exceed the amount of physical memory
  - The OS keeps those parts of the program currently in use in main memory and the rest on disk
  - The piece of the program being swapped between disk and memory as needed
  - Virtual memory can also work in a multiprogramming system
  - A technique that allows execution of a process without having to allocate it the memory space it needs, completely
  - It gives idea that large memory is available at programmer's disposal even computer has relatively small main memory
  - A program can be written assuming large address space is available
  - That is the virtual address space >> physical address space
  - The entire logical address space of a process must be in physical memory before the process can execute
  - First Implemented in Intel 80386

Virtual Memory

- Advantages of virtual memory:
  - Programs can be larger than the size of the physical memory
    - Virtual memory accommodates them
  - Virtual memory provides a logical view of memory which is larger than its physical size
- Virtual memory is not easy to implement
- May substantially decrease performance if it is used carelessly
- Virtual memory can be implemented via:
  - Demand paging
  - Demand segmentation

Virtual Memory

- Demand Paging
  - In demand paging, bring a page into main memory only when it is needed
    - Less I/O needed
    - Less memory needed
    - Faster response
    - More users
  - Page is needed ➔ reference to it
    - Invalid reference → abort
    - Not in memory → bring to memory
  - It can also be implemented in a segmentation system
- Demand Segmentation
  - Demand segmentation can also be used to provide virtual memory
  - Several systems provide paged segmentation scheme
    - Segments are broken into pages
  - The IBM OS/2 uses demand segmentation



Mechanism of Virtual Memory

- When a new process is initiated, the system loader must load one page (start point) into real memory
- Execution of the process will commence
- This will continue as long as memory references generated by this page also within the same page
- If the virtual address generated will reference a page which is not in real memory; this is termed as page fault, which generates an interrupt
- The demand paging will be used and system loader will load the next demanded page
- Execution will proceed after requested page is loaded
- By a series of page faults occurring in this way, a sub-set (resident set) of the pages of a process will gradually accumulate in real memory
- When the process terminates the OS releases all pages belonging to that process, making room for other processes

Mechanism of Virtual Memory

- Additional techniques
  - Hardware techniques to improve performance of virtual paged system
    - Translation look aside buffer (TLB)
    - Associative mapping

Effective Memory access time

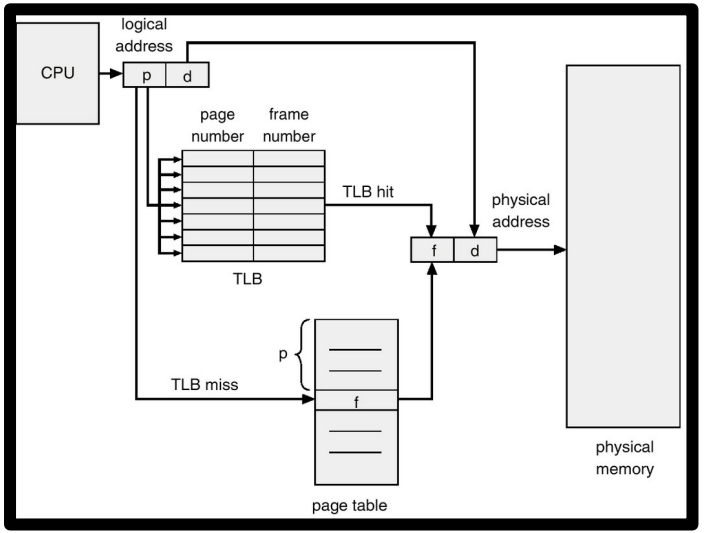
- Effective Access Time (EAT)
  - EAT = (1 – p) x memory access + p (page fault overhead + swap page out + swap page in)

- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- EAT = (1 – p) x 200 + p (8 milliseconds)
  = (1 – p x 200 + p x 8,000,000)
  If one access out of 1,000 causes a page fault, then EAT = 8.2 microseconds.
  This is a slowdown by a factor of 40!!
- If want performance degradation < 10 percent
  - 220 > 200 + 7,999,800 x p
  - 20 > 7,999,800 x p
  - p < .0000025
  - < one page fault in every 400,000 memory accesses

T(eff) = hit ratio * (TLB access time + Main memory access time) +
(1 – hit ratio) * (TLB access time + 2 * main memory time)
= 0.6*(10+80) + (1-0.6)*(10+2*80)
= 0.6 * (90) + 0.4 * (170)
= 122

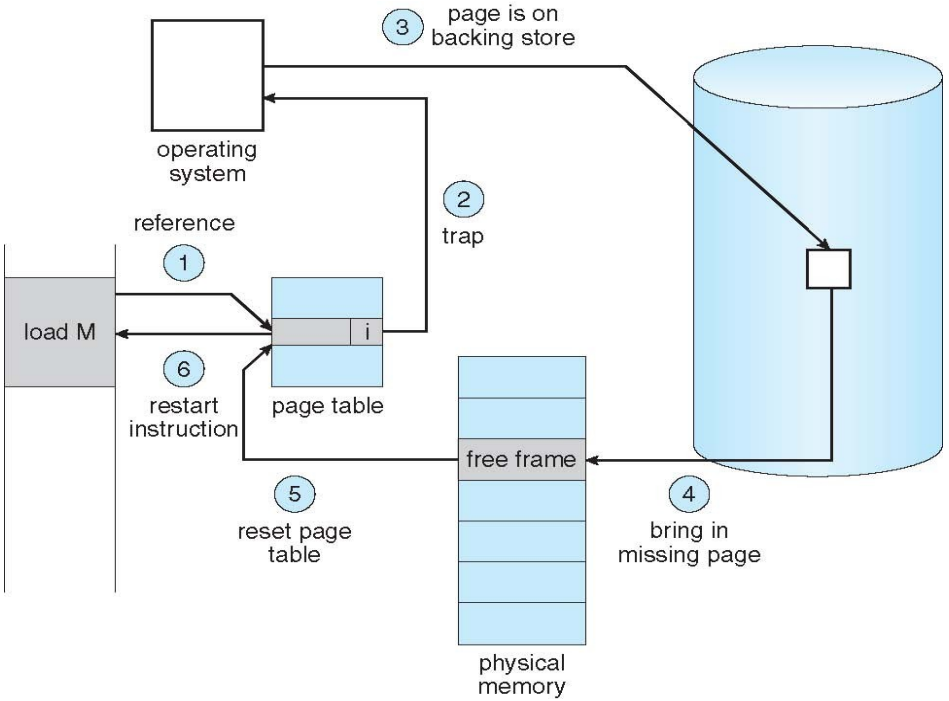
Translation Look aside Buffer

- A bit is used in each page table entry to indicate whether the corresponding entry is in main memory or not
- Each virtual memory reference can cause two physical memory accesses; one to fetch appropriate page table entry and one to fetch the desired data
- To reduce this overhead on system performance a memory cache TLB is used in most of the systems
- It is very fast and holds number of recently used page table entries
- The virtual address translation process will first search in TLB. If found, the real address can be formed immediately. If not, normal accessing of page table will proceed
- TLB holds limited number of page table entries, generally up to 32-4096
- Page table entry size is usually 4-8 bytes



Page Fault

- If the virtual address created a reference for a page which is not in real memory; it is termed as page fault
- Following measures should be done when a page fault occur
  - Get empty frame
  - Swap page into frame
  - Reset tables, present or valid bit = 1
  - Restart instruction



Page Replacement

- When a page fault occurs
  - The OS has to remove a page from memory to make room for incoming page
  - If the page to be removed has been modified it must be updated to disk
- For better performance, at each page fault
  - Pick a random page from memory that is not heavily used and replace it
  - If a heavily used page is removed, it may have to brought back in quickly, resulting in extra overhead
- Use modify (dirty) *bit* to reduce overhead of page transfers – only modified pages are written to disk

- Page replacement procedure
  - If no frame is free, we free a frame by writing its contents to disk
  - Change page table entry that the page is no longer in memory
  - Swap in new demanded page
  - Update the page table entry

Page Replacement

- There are different page replacement algorithms
- Each OS has its own page replacement technique
- The important thing is, it should be one with lowest page fault rate
- Replacement policy/strategy is concerned with selecting a page currently in memory to be replaced
- All policies are based on, the page that is removed should be the page least likely to be referenced in the near future
- **Principal of locality** says, there is a high correlation between recent referencing history and near future referencing patterns
- Most policies try to predict future behavior on the basis of past behavior

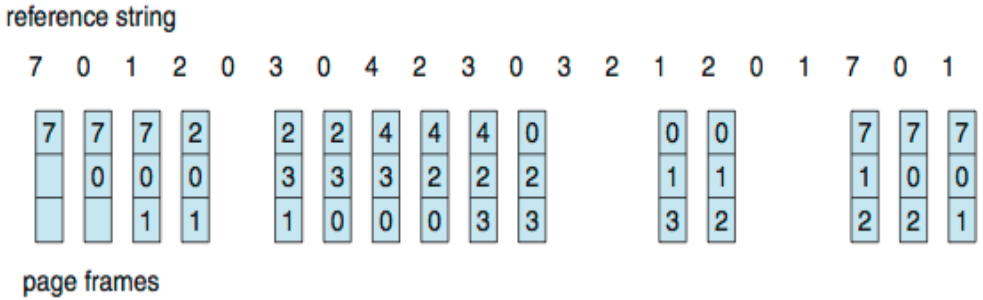
Page Replacement Algorithms

- There are different page replacement algorithms
- Each OS has its own page replacement technique
- The important thing is, it should be one with lowest page fault rate
- Replacement policy/strategy is concerned with selecting a page currently in memory to be replaced
- All policies are based on, the page that is removed should be the page least likely to be referenced in the near future
- **Principal of locality** says, there is a high correlation between recent referencing history and near future referencing patterns
- Most policies try to predict future behavior on the basis of past behavior

FIFO Algorithm

- The FIFO maintain a list of all pages in memory with the page at the head the oldest one and page at the tail most recent arrival
- On a page fault, the page at the head is removed and the new page added to tail of the list
- Salient points
  - FIFO algorithm is very easy to implement
  - Its performance is not always good because it may replace pages heavily used throughout the life of a program
  - Low overhead paging algorithm

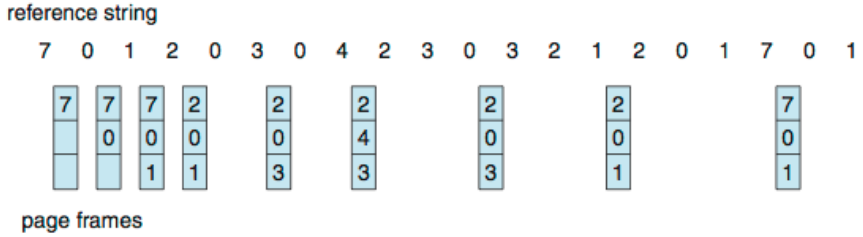
- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)



- Can vary by reference string: consider 1,2,3,4,1,2,5,1,2,3,4,5
- Adding more frames can cause more page faults!
- How to track ages of pages?
- Just use a FIFO queue

Optimal Algorithm

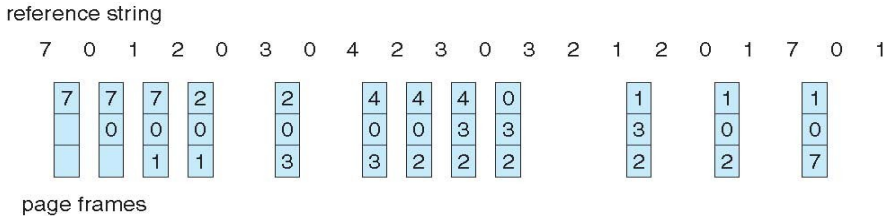
- Replace the page that will not be used for the longest period of time is called **principal of locality**
- This algorithm guarantees the lowest possible page fault rate
- Salient points
  - •Optimal replacement is much better than FIFO
  - •Difficult to implement, since it requires future knowledge of reference string
  - •Not used in practical systems because it cannot implement
  - •Optimal algorithm is mainly used for comparison studies



- Replace page that will not be used for longest period of time
  - •9 is optimal for the example
  - •How do you know this?
    - •Can’t read the future
  - •Used for measuring how well your algorithm performs

Least Recently Used (LRU) Algorithm

- Least recently used (LRU) algorithm chooses that page that has not been used for the longest period of time
- Salient points
  - LRU is quite good
  - Not easy to implement, the problem is to determine an order for the frames defined by the time of last use
  - Requires substantial hardware assistance because of tremendous overheads due to interrupts
- LRU feasible as a stack
  - Keep a stack of page numbers
  - Whenever a page is referenced, it is removed from the stack and put on top
  - In this way the top of the stack contains MRU page and bottom LRU page
- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page
- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?



Not Used Recently Algorithm

- Pages not used recently are not likely to be used in the near future and they may be replaced with incoming pages
- NUR strategy is implemented with the addition of two hardware bits per page
  - Reference bit
    - RB = 0 if the page has not been referenced
    - RB = 1 if the page has been referenced
  - Modified bit
    - MB = 0 if the page has not been modified
    - MB = 1 if the page has been modified
- Initially reference and modified bits are set to ‘0’
- As a reference to a particular page occurs, the reference bit of that page is set to ‘1’
- Whenever a page is modified its modified bit is set to ‘1’

- Whenever a page is to be replaced
  - It finds a page that has not been referenced
  - Otherwise it selects a referenced but unmodified page
  - Otherwise, it will replace a modified page that must be written back to the secondary storage
- There will be less overheads involved if algorithm finds a page in the lowest numbered group

Group 1	Unreferenced	Unmodified
Group 2	Unreferenced	Modified
Group 3	Referenced	Unmodified
Group 4	Referenced	Modified

- Which group seems impossible?

Random Page Replacement Algorithm

- Choose random page and replace it
- This strategy could select any page for replacement, including the next page to be referenced
- Random page replacement decisions can be made quickly
- This scheme is rarely used

Least Frequently Used (LFU) Algorithm

- LFU page replacement algorithm replaces the page from memory which is least frequently used
- It keeps a counter for the number of references to each page and the page with the smallest count is replaced
- This algorithm suffers from the situation in which a page initially used heavily, but then never again used

Most Frequently Used (MFU) Algorithm

- MFU page replacement algorithm replaces the page from memory which is most frequently used
- It is based on the argument that the page with the smallest count was probably just brought in and has yet to be used
- Its implementation is expensive
- Neither MFU nor LFU replacement algorithm is common

Allocation of Frames

- Each process needs ***minimum*** number of frames
- Example: IBM 370 – 6 pages to handle SS MOVE instruction:
  - instruction is 6 bytes, might span 2 pages
  - 2 pages to handle *from*
  - 2 pages to handle *to*
- ***Maximum*** of course is total frames in the system
- Two major allocation schemes
  - fixed allocation
  - priority allocation
- Many variations

Fixed Allocation

- Equal allocation – For example, if there are 100 frames (after allocating frames for the OS) and 5 processes, give each process 20 frames
  - Keep some as free frame buffer pool
- Proportional allocation – Allocate according to the size of process
  - Dynamic as degree of multiprogramming, process sizes change

Priority Allocation

- Use a proportional allocation scheme using priorities rather than size

- If process P_i generates a page fault,
  - select for replacement one of its frames
  - select for replacement a frame from a process with lower priority number

Global vs. Local Allocation

- Global replacement
  - Process selects a replacement frame from the set of all frame
  - one process can take a frame from another
- Local replacement
  - Each process selects from only its own set of allocated frames

Non-Uniform Memory Access

- So far all memory accessed equally
- Many systems are **NUMA** – speed of access to memory varies
  - Consider system boards containing CPUs and memory, interconnected over a system bus
- Optimal performance comes from allocating memory “close to” the CPU on which the thread is scheduled
  - And modifying the scheduler to schedule the thread on the same system board when possible
  - Solved by Solaris by creating **lgroups**
    - Structure to track CPU / Memory low latency groups
    - Used my schedule and pager
    - When possible schedule all threads of a process and allocate all memory for that process within the lgroup

Thrashing

- If a process does not have “enough” pages, the page-fault rate is very high
  - Page fault to get page
  - Replace existing frame
  - But quickly need replaced frame back
  - This leads to:
    - Low CPU utilization
    - Operating system thinking that it needs to increase the degree of multiprogramming
    - Another process added to the system
- **Thrashing** ≡ a process is busy swapping pages in and out



Thank you